CO2-AT3_Questions-Slot-C

Register No: 192425303

Student: NANUBALA MADHUSUDHAN

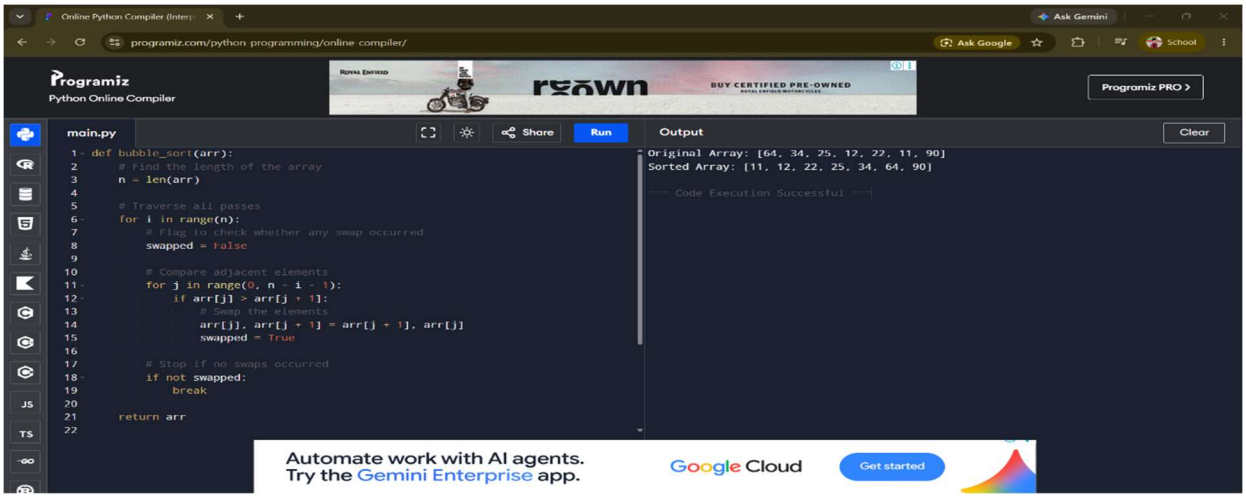
Assigned Question:

Q22. Debugging Bubble Sort Code (Missing Early Termination Optimization)

The following Bubble Sort code eventually produces the correct output, but it performs unnecessary passes even when the array becomes sorted early. Carefully analyze the existing implementation and identify the inefficiency in loop execution. Apply debugging and optimization techniques to introduce a flag-based termination mechanism. Explain how this optimization improves the practical performance of the algorithm. Analyze the corrected version in best, average, and worst cases. Rewrite the final optimized code with proper comments and readable structure.

```
def bubble_sort(arr):
  n = len(arr)
  for i in range(n):
    for j in range(0, n-i-1):
      if arr[j] > arr[j+1]:
        arr[j], arr[j+1] = arr[j+1], arr[j]
  return arr
```

Solution:



1. Missing Early Termination Optimization

Given Code:

```
for i in range(n):
```

Mistake:

* The algorithm continues all passes even after the array becomes sorted.

* This causes unnecessary comparisons and increases execution time.

Correction:

swapped = False

2. Missing Early Termination Condition

Given Code:

for i in range(n):

Mistake:

* The algorithm does not check whether any swapping occurred during a pass.

* Even if the array is already sorted, it continues executing the remaining passes.

Correction:

```
for i in range(n):
    swapped = False
```

3. Missing Swap Tracking

Given Code:

```
if arr[j] > arr[j + 1]:
    arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

Mistake:

* The code swaps elements correctly but does not record whether a swap has occurred.

* Without tracking swaps, the algorithm cannot terminate early.

Correction:

```
if arr[j] > arr[j + 1]:
    arr[j], arr[j + 1] = arr[j + 1], arr[j]
    swapped = True
```

4. Missing Break Statement

Mistake:

* The algorithm performs unnecessary passes even when no elements are exchanged.

* This reduces the practical performance of Bubble Sort.

Correction:

if not swapped:

    break

5. Readability Improvements

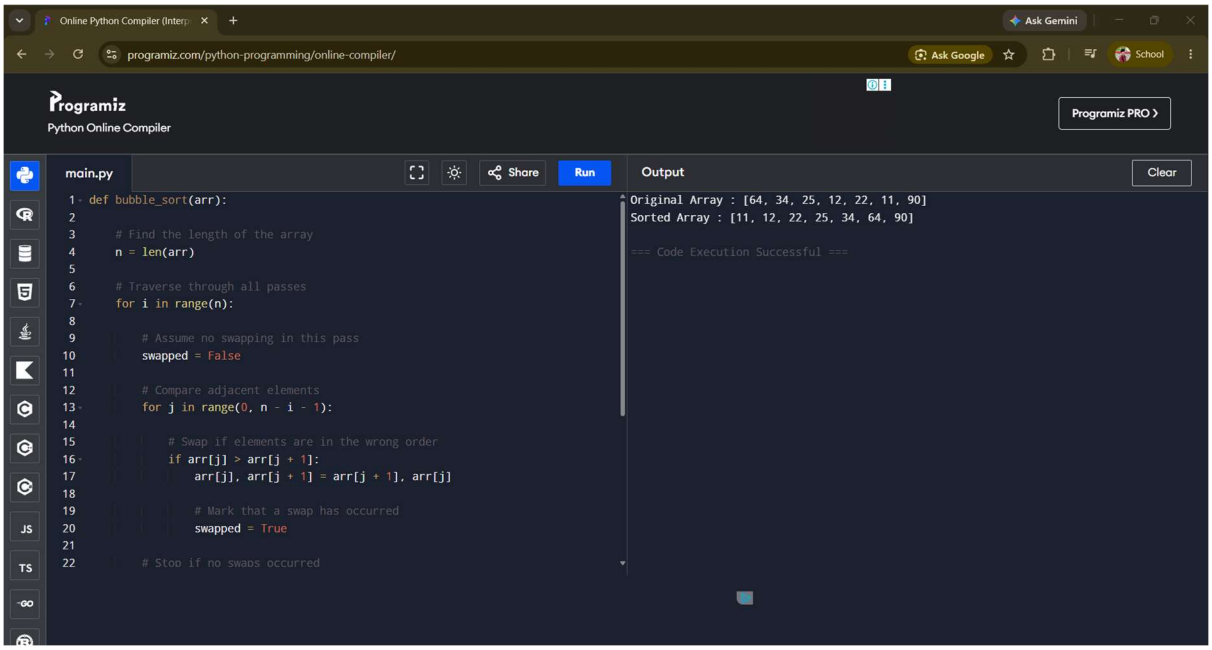
* Add comments explaining each step.

* Use proper indentation.

* Use meaningful variable names.

* Keep the code simple and readable.

After correction:



Why the Optimization Improves Performance

The optimized Bubble Sort uses a swapped flag to detect whether any exchange occurs during a pass. If no elements are swapped, the array is already sorted. The algorithm terminates immediately instead of executing the remaining passes. This reduces unnecessary comparisons and improves the practical performance of Bubble Sort.

Time Complexity

- Best Case: O(n)
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Space Complexity: O(1)